



Cellular Automata-based Toxicity Analysis for Social Media Comments

M. J. Elizabeth²(✉)  and Raju Hazari¹ 

¹ Department of Computer Science and Engineering, National Institute of Technology, Calicut, Kerala, India

² Department of Computer Science, Mary Matha Arts and Science College, Mananthavady, Wayanad, Kerala, India
elizabethmj@marymathacollege.ac.in

Abstract. Toxic analysis of social media data aims to identify harmful language, including hate speech, bullying, and abusive comments, to promote healthier online environments. This paper presents a cellular automata-based approach for toxic text classification. The results show that it has the ability to capture text context and patterns through simple, localized interactions. Our proposed model encodes text data using Unicode formats, which are used as rules for the CA evolution process. The CA signals are then used as input for machine-learning models. This hybrid approach enables the classification model to detect nuanced toxic expressions. Experimental results on benchmark datasets demonstrate that the CA-based toxic analysis model achieves comparatively good accuracy in identifying class labels of toxic and non-toxic comments.

Keywords: Cellular Automata (CA) · Toxic Comments · Hate Speech · Machine Learning · Text Classification

1 Introduction

Toxic text classification has improved greatly in recent years, driven by increased user-generated data on social media platforms. However, this large amount of online content also brings new challenges. Hateful, harassing, and harmful language in texts can seriously harm online communities. Toxic text must be automatically identified and categorized to provide a more secure and welcoming online community. People and communities are seriously threatened by harmful material, which includes hate speech, cyberbullying, and other types of online harassment. It is important to recognize and minimize toxic remarks since they can have serious effects in real life and make it harder to have constructive conversations. Frequently, these remarks are impolite, hostile, nasty, or cruel. Toxic analysis helps us to reduce such hateful words from the text as well as it has potential applications across multiple industries for brand monitoring, market research, and social media analysis, and is also used to analyze employee feedback. It can be used in healthcare sectors for psychological analysis and patient

feedback monitoring. Threat detection is another application area by analyzing the toxicity of the text and behaviors in security footage or online communications. Additionally, text toxicity analysis is essential for creating emotion-aware interactions that require understanding the user's emotions. However, toxic text analysis presents several challenges, such as modeling the context of conversations, addressing the dynamic shifts in offensive language among participants, and amplifying the complexity of the task.

Numerous studies have already generated favorable results in this domain. In 2009, supervised learning methods were used for detecting harassment [36]. A well-structured survey [31] was done in 2017 in which multiple application areas of hate speech classification, different classification techniques, challenges, and an overview of data considerations were explained in detail. Another survey was done by Fortuna et.al [16] in 2018. Various machine-learning (ML) methods were introduced in that paper. Logistic Regression [8], Random Forest, and SVM [19], and many other ML models [3, 5, 6] were analyzed for toxicity analysis. A variety of prominent deep learning models [3] are also applied in the field of text classification tasks such as Convolutional Neural Networks (CNNs) [3], Recurrent Neural Networks (RNNs) [27], Deep Averaging Networks (DANs), Denoising Autoencoders (DAEs), Long Short-term Memory (LSTM) [10] networks, Bi-directional Long Short-term Memory (Bi-LSTM) networks, Gated Recurrent Units (GRUs), and attention mechanisms. In recent years, pre-trained language models [37] such as BERT, RoBERTa, XLM, and Glove. have come into play for text classification. Their ability to capture bidirectional context and nuances in language has led to promising results. Many researchers combine multiple of these methods within a single model to enhance the performance of toxic text analysis. All these approaches completely rely on conventional language-related features, such as TF-IDF, POS, BOW, N-grams, sentiment, hashtags, mentions, retweets, URLs, character or word counts, syllables, dictionaries, syntactic and semantic features, and more. Our primary goal is to develop a machine-learning model without relying on such language-specific features for toxicity analysis.

The proposed method entails bringing the concept of cellular automata (CA) into the field of natural language processing. The CA is used to evaluate the complexities of every sentence through a weight-based approach in which word weight is calculated based on the occurrence of the words in the text along with deep learning models. We have used publicly available English, Hindi, and Bengali hate speech datasets for this analysis. The proposed method is a language-independent Deep Learning (DL) Model for toxic text or hate speech classification. In recent works, we have used CA-based machine learning models [11–15] for sentiment analysis which gives comparatively good results. This work motivates us to use the CA-based machine-learning model for the toxicity classification of texts.

The proposed approach is a DL model, implemented using the combination of cellular automata (CA) and deep-learning models. The process begins by transforming each comment in the dataset into a vectorized form using Unicode formats, which are then utilized as rules for the CA model. Through an iterative evolution process, the Cycle Length (CL) values of each CA cell are determined. Those CL values are further used for calculating the weights of each word in the

text. Finally, we will get a weight matrix that is used as an input for DL models. Ultimately, the model predicts toxicity, categorizing them into two classes such as positive and negative classes.

The paper is organized as follows. The basics of cellular automata are described in Section 2. Section 3 describes the design techniques of the CA model. In Section 4, the results and its evaluation details are explained. Finally, Section 5 concludes the work.

2 Basics of Cellular Automata

The history of Cellular Automata traces back to Von Neuman’s work [26], where they initially developed a CA-based model to simulate a biological self-reproduction system. Cellular Automata (CA) are discrete, spatially extended dynamical systems that have been extensively explored as models for various physical systems. They evolve within discrete space and time, featuring a lattice of cells. Each cell stores a discrete variable representing its current state at time ‘ t ’. The state of a cell at ‘ $t + 1$ ’ is determined by its current state and the states of its neighbouring cells at a time ‘ t ’. The simplest form of CA is Elementary Cellular Automata (ECA), introduced by Wolfram [35]. ECA are fundamental and straightforward models for understanding complex behaviors that can emerge from simple local rules. In simpler terms, ECA are cellular automata where the future states of each cell depend on the current state of that cell and its two closest neighbours. These cells can be thought of as binary (typically 0 or 1), having two possible values, and the interactions between neighbouring cells lead to the evolution of the system over discrete time steps. The state of a cell at time ‘ $t + 1$ ’ is determined as:

$$S_i^{t+1} = f(S_{i-1}^t, S_i^t, S_{i+1}^t) \quad (1)$$

where ‘ f ’ represents the next state function, and the variables S_{i-1}^t , S_i^t and S_{i+1}^t correspond to the present states of the left, self, and right neighbour of the i^{th} cell at time t . The function $f : \{0, 1\}^3 \mapsto \{0, 1\}$ can be expressed as a look-up table (see Table 1) with 8 possible combinations of inputs, each resulting in an output of either 0 or 1. There are a total of 256 possible elementary cellular automata rules, and Table 1 illustrates two examples: Rule 120 and Rule 240.

In the traditional setup of a Cellular Automata (CA), all the cells within the automata follow the same next state function, making it a “uniform” CA. However, in some instances, a more flexible approach is adopted where individual cells can follow different next-state functions or rules. This variant of CA is referred to as a “non-uniform” or “hybrid” CA. In this study, we chose to use a non-uniform elementary cellular automata configuration under periodic boundary conditions where the first and last cells are considered neighbours of each other. This non-uniform approach provides greater complexity and flexibility in modeling and simulating the system, allowing for a broader range of behaviors and patterns to emerge within the CA that helps to learn the context of text.

Table 1. Look-up table for rules 120 and 240

Present State :	111	110	101	100	011	010	001	000	Rule
	(7)	(6)	(5)	(4)	(3)	(2)	(1)	(0)	
(i) Next State :	0	1	1	1	1	0	0	0	120
(ii) Next State :	1	1	1	1	0	0	0	0	240

3 Proposed CA Model Implementation

In the proposed classification framework, we’ve created a weight-based classifier with a deep-learning model designed to assess whether the text is toxic or not. In this context, the input text is transformed into a vector format with the help of Unicode standards. Our approach involves building a language-independent machine-learning classifier based on CA. The design of our model can be categorized into two modules, which are listed below:

- Module-I: Weight Matrix Generation
- Module-II: Toxicity Prediction Process

3.1 Module-I: Weight Matrix Generation

We need to perform the data preprocessing process, text vectorization process, and the CA evolution process before creating the weight matrices.

3.1.1 Data Preprocessing

Each comment in the dataset is preprocessed using the following steps:

1. Remove extra spaces, punctuation, and new lines.
2. Remove repeated patterns, URLs, and user tags.
3. Remove single characters, emojis, and stop words from the comments.
4. Remove non-English characters from English comments, Non-Hindi characters from Hindi comments, and Non-Bengali characters from Bengali comments.
5. Remove the start and end white spaces.
6. Convert each comment into lowercase letters for English comments.

3.1.2 Text Vectorization

The preprocessed data is converted into a vectorized form using Unicode values of every character found in the text. The process for creating CA rule vectors involves the following steps:

- Identify the Unicode position for each character from the Unicode charts.

- If the Unicode position is represented in the format: 0x (p1 p2 p3 p4), where p1, p2, p3, p4 are hexadecimal characters, then the decimal value derived for the character is calculated with the expression: $256^1(16p1 + p2) + 256^0(16p3 + p4)$. Subsequently, the corresponding CA rule can be calculated using the expression: $\langle (16p1 + p2), (16p3 + p4) \rangle$.
- If p1 and p2 both are '0' then the corresponding CA rule calculated using the expression is: $\langle (16p3 + p4) \rangle$

A few examples are given below:

- The Unicode of the English character 'f' is 0x(0066). Here p1=0, p2=0, p3=6 and p4=6. The CA Rule is determined using the formula:

$$CA\ Rule = \langle (16p1 + p2), (16p3 + p4) \rangle \quad (2)$$

Substituting the values: CA Rule = $\langle (16*0 + 0, 16*6 + 6) \rangle = \langle 0, 102 \rangle$.

Thus, the CA Rule for the character 'f' is represented as [102].

- The Unicode of the Hindi character 'फ' is 0x(092B). Here p1=0, p2=9, p3=2 and p4=B. The CA Rule is determined using the Eq. 2. Substituting the values: CA Rule = $\langle (16*0 + 9, 16*2 + B) \rangle = \langle 9, 43 \rangle$.

Thus, the CA Rule for the character 'फ' is represented as [9, 43].

- The Unicode of the Bengali character 'ক' is 0x(0995). Here the CA Rules will be [9,149].

If we apply it to words then,

- Let's consider a word 'love' which has 4 letters: ['l', 'o', 'v', 'e'] and their Unicode characters are [0x(006C), 0x(006F), 0x(0076), 0x(0065)]. The corresponding CA rules are [108, 111, 118, 101]

Examples of different texts and their corresponding rule vectors are given below:

English Text- "many unsecured wi fi hot spots so many idiot wikipedia pages"

Rule Vectors - [109, 97, 110, 121, 32, 117, 110, 115, 101, 99, 117, 114, 101, 100, 32, 119, 105, 32, 102, 105, 32, 104, 111, 116, 32, 115, 112, 111, 116, 115, 32, 115, 111, 32, 109, 97, 110, 121, 32, 105, 100, 105, 111, 116, 32, 119, 105, 107, 105, 112, 101, 100, 105, 97, 32, 112, 97, 103, 101, 115]

Hindi Text- "देखिए मेरे साथ अभी अबकी बार भोजपुरी सितारों की धारठीक बजे" (English Translation: "Watch with me this time the edge of Bhojpuri stars at the right time.")

The characters are separated into individual list items, making it possible to analyze each one independently. The output of character tokenization is given below:

['द', 'े', 'ख', 'ि', 'ए', 'म', 'े', 'र', 'े', 'स', 'ा', 'थ', 'अ', 'भ', 'ी', 'अ', 'ब', 'क', 'ी', 'ब', 'ा', 'र', 'भ', 'ो', 'ज', 'प', 'ु', 'र', 'ी', 'स', 'ि', 'त', 'ा', 'र', 'ो', 'ं', 'क', 'ी', 'ध', 'ा', 'र', 'ठ', 'ी', 'क', 'ब', 'ज', 'े', '']

Rule Vectors - [9, 38, 9, 71, 9, 22, 9, 63, 9, 15, 32, 9, 46, 9, 71, 9, 48, 9, 71, 32, 9, 56, 9, 62, 9, 37, 32, 9, 5, 9, 45, 9, 64, 32, 9, 5, 9, 44, 9, 21, 9, 64, 32, 9, 44, 9, 62, 9, 48, 32, 9, 45, 9, 75, 9, 28, 9, 42, 9, 65, 9, 48, 9, 64, 32, 9, 56, 9, 63, 9, 36, 9, 62, 9, 48, 9, 75, 9, 2, 32, 9, 21, 9, 64, 32, 9, 39, 9, 62, 9, 48, 9, 32, 9, 64, 9, 21, 32, 9, 44, 9, 28, 9, 71, 32, 9, 100]

Bengali Text- “আপনাদের ওপর শরম লাগে এখানে মারছে আর আপনি ভিডিও বানাচ্ছেন.”
(English Translation: “Shame on you guys here and making videos”)

Rule Vectors - [9, 134, 9, 170, 9, 168, 9, 190, 9, 166, 9, 199, 9, 176, 32, 9, 147, 9, 170, 9, 176, 32, 9, 182, 9, 176, 9, 174, 32, 9, 178, 9, 190, 9, 151, 9, 199, 32, 9, 143, 9, 150, 9, 190, 9, 168, 9, 199, 32, 9, 174, 9, 190, 9, 176, 9, 155, 9, 199, 32, 9, 134, 9, 176, 32, 9, 134, 9, 170, 9, 168, 9, 191, 32, 9, 173, 9, 191, 9, 161, 9, 191, 9, 147, 32, 9, 172, 9, 190, 9, 168, 9, 190, 9, 154, 9, 205, 9, 155, 9, 199, 9, 168]

3.1.3 CA Evolution and CL-Graph Generation

The CA evolution process is a sequence of state transformations driven by rules applied iteratively to a grid of cells. Each vector in the rule vector table is used as a rule for the CA evolution process, which is carried out to generate cycle-length (CL) values. The process begins with an initial configuration, representing the starting state of all cells in the grid. This configuration can be random (e.g., cells randomly initialized as 0 or 1) or structured (e.g., alternating 0's and 1's or a specific pattern). The next state of each cell depends on the states of its neighbourhood. In ECA, the neighbourhood includes the current cell and its immediate left and right neighbours (radius 1). For a 2-D CA, the common neighbourhood structures include Moore (8 neighbours) or Von Neumann (4 neighbours) configurations. The rule vector defines how the states of a cell and its neighbours determine the next state. The rule can be encoded as a single number, e.g., Rule 30 or Rule 110 in Wolfram's notation. The CA evolves by applying the rule iteratively over a predefined number of steps or until a stopping condition is met. The local dependency and the deterministic nature of CA make them suitable for modeling complex phenomena. During the evolution, a repeating pattern or cycle may emerge. The length of the repeating pattern is called the Cycle Length (CL) value. If no repeating pattern is observed within a threshold of iterations, the CL value can be assigned as a negative value or zero. By iteratively applying simple rules to the states of cells and their neighbours, CA demonstrate rich and diverse behavior, making them a powerful computational model for exploring complex systems. The steps used to generate CL values are given below:

1. *Initialization Stage*: The initial state of the ECA alternates between 1 and 0 for each cell, creating a regular pattern such as 101010... or 010101... The length of the initial configuration matches the size of the rule vector. The same pattern is used for each text in the study.
2. *State Transition*: The current state of each cell in the ECA is updated at the time of transition by using its current state and its neighbouring states. These transitions are governed by the rule number specified in the rule vector.

- 3. *Iteration Process*: ECA undergoes iterative state transitions, evolving over discrete time steps. The iteration value is a critical parameter, determining how long the CA evolves before stopping. The minimum number of iterations is set to ‘1000’ with periodic boundary conditions applied. If the length of the text goes beyond ‘2000’ then half of the length of the text is considered as iteration step. For instance, if the maximum length of the comments in the dataset is ‘2400’, the iteration value is set to ‘1200’.
- 4. *Emergence of Patterns*: After a specific number of iterations, a repeating pattern may emerge. The Cycle Length (CL), representing the periodicity of these patterns, is a widely used metric for characterizing CA dynamics. Identifying the CL helps quantify the complexity and structure of the text when CA is applied to text data analysis, as observed in various pattern recognition tasks.
- 5. *Threshold Evaluation*: A threshold of ‘32’ repetitions is set, inspired by prior studies that suggest recurring patterns below a certain limit may represent noise or transient states rather than meaningful cycles [20]. If the recurrence does not occur or fails to reach the threshold of 32 repetitions, then the CL value is taken as ‘0’ or any negative value to denote non-periodicity.

The visualization of the evolution is shown in Fig. 1 in which the specific patterns of variable length recur periodically, as indicated by the bold lines. To illustrate this further, we provide an example that describes the input text and the corresponding CL values. From these CL values, we can generate CL graphs, which are displayed in Figs. 2, 3 and 4.

.....	1	0	1	0	1	0	1	0	
.....	0	1	0	1	0	1	1	0	
.....	1	0	1	0	1	1	1	0	
.....	1	0	0	1	0	0	1	1	
.....	0	0	1	0	0	1	1	1	
.....	1	1	0	1	1	0	1	0	
.....	1	1	0	0	1	0	0	0	
.....	0	0	0	1	1	0	0	1	
.....	1	1	0	0	0	1	1	0	
.....	1	0	0	1	1	0	0	0	
.....	0	1	1	0	0	0	0	0	
.....	1	0	0	1	0	0	0	0	
.....	1	0	0	0	0	1	0	1	
.....	:	:	:	:	:	:	:	:	

Fig. 1. CA Evolution.

English Text: “many unsecured wi fi hot spots so many idiot wikipedia pages”
CL values of English Text: [8, 8, 8, 8, 1, 1, 1, 6, 6, 6, 2, 2, 1, 1, 1, 2, 1, 2, 1, 1, 1, 1, 4, 4, 2, 2, 6, 6, 3, 1, 1, 2, 18, 18, 18, 18, 18, 18, 3, 3, 1, 1, 1, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 1, 1, 4, 4, 4, 2]

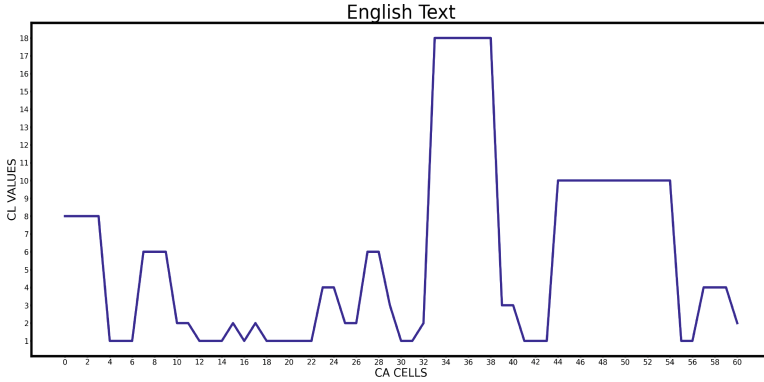


Fig. 2. CL-graph for English Text.

Hindi Text- “देखिए मेरे साथ अभी अबकी बार भोजपुरी सितारों की धारठीक बजे ः” (English Translation: “Watch with me this time the edge of Bhojpuri stars at the right time.”)

CL values of Hindi Text: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 6, 6, 6, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 4, 2, 2, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 5, 5, 5, 5, 1]

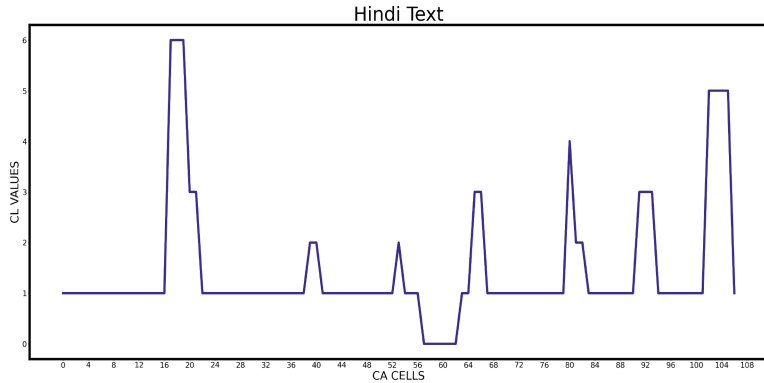


Fig. 3. CL-graph for Hindi Text.

Bengali Text- “আপনাদের ওপর শরম লাগে এখানে মারছে আর আপনি ভিডিও বানাচ্ছেন.” (English Translation: “Shame on you guys here and making videos”)

CL values of Bengali Text: [3, 3, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 9, 9, 9, 3, 3, 3, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1, 6, 3, 3, 3, 3, 3, 1, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1]

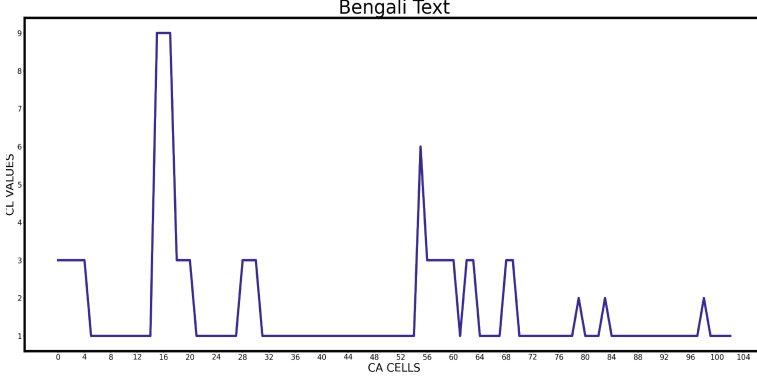


Fig. 4. CL-graph for Bengali Text.

3.1.4 Word Weight Generation Procedure

The proposed model identifies each unique word within the dataset and then we need to find out the CL sequences of each word. These CL sequences at sentence and word levels are used to determine the CL-Weight for each word. The steps for calculating CL-Weight are as follows:

- **Sentence CL-Sum (X):** The sum of CL values of a sentence.
- **Word CL-Sum (Y):** Sum of the CL values of the word.
- **CL-Weight:** Let α represent a word that appears in n number of sentences within the dataset. If the Word CL-Sum of α is Y_α , then the CL-Weight of α is calculated by:

$$CL - Weight_\alpha = \frac{\sum_{i=1}^n X_i}{Y_\alpha}$$

The weights of each word are used to generate a weight matrix for each comment in the dataset, serving as input to the deep learning models. For example, let us consider a Bengali dataset with only two sentences then, the procedure of word vectorization is as follows:

Text 1 - “দারুন মজার নাটক..” (English Translation: “Great fun drama”)

CL Values - [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1] and the sentence CL-Sum is 29.

Text 2 - “নাটক অনেক ভালো লাগছে খুবি মজার ছিলো” (English Translation: “The drama is very good and very funny”)

CL Values - [4, 1, 9, 9, 9, 9, 3, 3, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 4, 2, 1, 1, 1, 4, 4, 4] and the sum of CL values is 123.

Each word in Text 1 and Text 2 is given in Tables 2 along with its CL values and corresponding word weights. Tables 3 shows Text 1 and Text 2 in vector form.

Table 2. CL values, Word CL-Sum and CL-Weight of words

Unique_Words	CL_Values	Word_CL-Sum	CL-Weight
দারুন	[1,1,1,1,1,1,1,1,1]	10	29/10=3
মজার	[1,1,1,1,1,1,1,1]	8	(29+123)/8 = 19
নাটক	[1,1,1,1,1,1,1,1,1]	10	(29+123)/10 = 15
অনেক	[1,1,1,1,1,1,1,1]	9	123/9=14
ভালো	[1,1,1,1,1,0,0,0]	5	123/5=25
লাগছে	[1,1,1,1,1,1,1,2,1]	12	123/12=10
খুবি	[1,1,0,0,0,1,1,1,1]	6	123/6=21
ছিলো	[1,2,1,1,1,0,0,1]	7	123/7=18

Table 3. Mappings of Reviews

Reviews	Word Vectors
“ দারুন মজার নাটক ”	[3,19,15]
“নাটক অনেক ভালো লাগছে খুবি মজার ছিলো ”	[15,14,25,10,21,19,18]

4 Toxicity Prediction Process

In this section, we explored the experiment results, performance analysis, and comparative study with other state-of-the-art models. We have used four DL models such as LSTM, Bi-LSTM, CNN, and CNN-LSTM. We used 64 LSTM layers, 128 convolution layers, and 256 Bi-LSTM layers. The experiment is done with ‘adam’ as the optimizer with learning_rate=0.0005. We set both dropout and recurrent dropout rates to 0.2. The ‘Relu’ activation function is used in all cases. The architecture of CNN and Bi_LSTM models are shown in Figs. 5 and 6.

4.1 Datasets Description

- *Hate Speech and Offensive Language Dataset* [2]: The dataset is created using Twitter data for hate-speech detection. There are 24K comments classified as hate speech, offensive language, and neither (class labels: 0 - hate speech, 1 - offensive language, and 2 - neither offensive nor non-offensive).
- *Jigsaw Toxic Comment Classification Dataset* [7]: The dataset contains 1,59,571 comments from Wikipedia’s talk pages that have been labeled by human raters as toxic or non-toxic. The types of toxicity are toxic, severe_toxic, obscene, threat, insult, and identity_hate.
- *Hindi Dataset* [24]: Hasoc task A dataset is used for the binary classification of Hate and Not Offensive Hindi texts. The dataset contains 2,963 training data and 663 texts for testing the performance measures.
- *Bengali Dataset* [30]: The dataset has 15,000 comments classified as hate speech and Non-offensive.

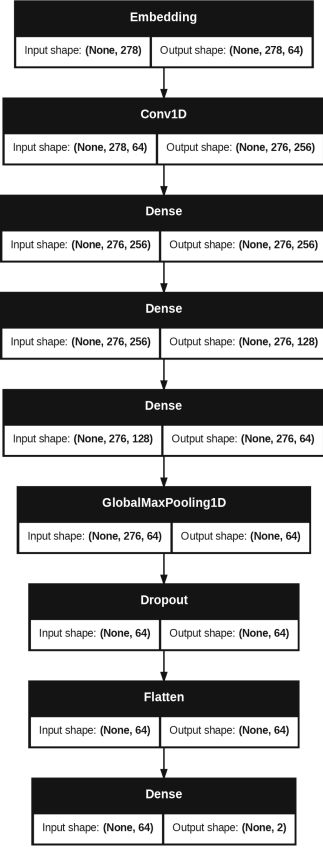


Fig. 5. CA-based CNN Model.

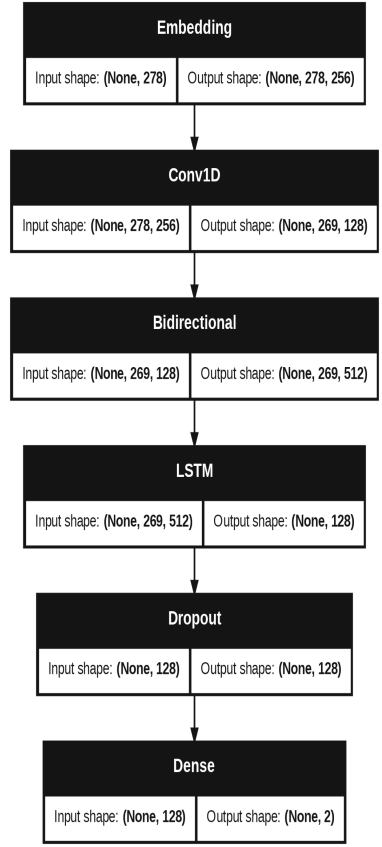


Fig. 6. CA-based Bi-LSTM Model.

4.2 Results and Performance Analysis

The results of the proposed approach are discussed with the help of accuracy, precision, recall, and F1_Score values. The F1_Score is computed using precision and recall, utilizing a confusion matrix comprising elements such as true positives, true negatives, false positives, and false negatives. This F1_Score serves as a measure to assess the classification accuracy of the model developed. CNN and LSTM achieved the overall best results but LSTM had relatively worst accuracy and F1_Score for the Hindi dataset. The overall best result is obtained with CNN for three cases except the Jigsaw dataset. LSTM produces the highest accuracy and an F1_Score of 93.73% and 92.82%, respectively, for the Jigsaw dataset. The results are given in Table 4.

4.3 Comparative Analysis

The proposed model produces approximately 92% accuracy for English hate speech datasets and it is comparable with other state of art models. The result analysis is given in Table 5 for English toxicity analysis. Our model produces better results than all the state-of-art models for the Hate and Offensive Language dataset. Table 6 shows the comparative analysis of non-English Indian languages such as Hindi and Bengali toxic text datasets. Very few works are available for Hindi and Bengali toxic text classification tasks. We have got comparatively good results for both of the languages. From Tables 5 and 6, we can conclude that the model produces more than 90% for all performance analysis metrics for English languages and approximately 85% for non-English languages. The lower accuracy of non-English languages compared to English can largely be attributed to the disparity in dataset sizes. English benefits from vast and diverse datasets, such

Table 4. Results of proposed approach for different toxic datasets

Model Used	Accuracy	Precision	Recall	F1_Score
Jigsaw Dataset [7]				
LSTM	93.73	92.80	93.72	92.82
Bi-LSTM	91.18	91.78	91.18	91.46
CNN	92.36	91.29	92.36	91.68
CNN_LSTM	93.54	92.50	93.54	92.35
Hate Speech and Offensive Language Dataset [2]				
LSTM	89.37	89.23	89.36	88.90
Bi-LSTM	86.60	87.15	86.59	85.27
CNN-LSTM	87.49	87.16	87.48	86.93
CNN	90.62	90.55	90.61	90.25
Hindi [24]				
LSTM	76.20	76.73	76.2	75.93
Bi-LSTM	78.26	78.57	78.27	78.26
CNN-LSTM	77.12	77.11	80.43	78.72
CNN	81.01	81.77	81.00	80.98
Bengali [30]				
LSTM	83.80	83.55	83.6	83.47
Bi-LSTM	83.33	83.06	83.34	82.96
CNN-LSTM	82.07	82.26	82.06	82.15
CNN	85.20	85.00	85.20	85.02

as the Jigsaw dataset, IMDB reviews, SQuAD, and Wikipedia dumps, which are well-labeled and extensively used to train deep learning models. In contrast, non-English languages often have smaller datasets, limiting the linguistic patterns that models can learn. Additionally, non-English datasets may suffer from inconsistent labeling and a lack of representation for regional dialects, further reducing model performance.

Table 5. Different models and their results in terms of F1_score for English hate speech datasets

Authors	Techniques used	Accuracy (%)	F1_score (%)
Jigsaw Dataset[7]			
Garg et al. (2024) [17]	Explainable HSD system	93.60	98.19
Singh et al. (2022) [32]	AlexNet based on CNN	98.50	79.00
ashok et al. (2021) [21]	MCBiGRU	-	71.70
Wang et al. (2021) [34]	CNN with GloVe	-	99.50
Zhao et al. (2021) [37]	RoBERTa	-	78.22
Georgakopoulos et al. (2018) [18]	CNN	> 90.00	91.20
Aken et al. (2018) [1]	Ensemble	-	79.10
Our model	CA based LSTM	93.73	92.82
Hate and Offensive Language [2]			
Mamun et al. (2024) [23]	BERT + LIME	74.92	69.83
Nandini et al. (2022) [25]	BERT + LIME	82.60	82.80
Rajput et al. (2021) [28]	Bi-LSTM	80.15	79.71
Mandl et al. (2020) [24]	LSTM	-	83.95
Davidson et al. (2017) [9]	Logistic regression with L2 regularization	82.33	90.00
Our model	CA-based CNN Model	90.62	90.25

Table 6. Comparative analysis of toxic datasets for Non-English Indian languages

Model Used	Accuracy	Precision	Recall	F1_Score
Hindi [24]				
CNN (2021)[33]	78.00	85.00	76.00	76.00
RoBERTa (2021)[33]	80.00	77.80	75.40	76.30
CNN (2020) [4]	82.00	84.00	82.00	81.00
CNN (2020) [24]	-	-	-	82.02
Proposed Method(CNN)	81.01	81.77	81.00	80.98
Bengali [30]				
SVM (2021) [30]	87.50	-	-	91.10
SVM -U (2022) [29]	-	88.9	88.7	88.7
Bengali BERT (2020) [22]	70.60	70.70	70.60	70.50
CA-based CNN	85.20	85.00	85.20	85.02

5 Conclusion and Future Work

In this work, we proposed to use cellular automata-based machine learning (CAML) to perform the toxic text classification task. The training of data relies on a weight-based approach using CA with deep-learning models. The results show that CA acquire an understanding of the context of text through its evolutionary process from the vectorized texts using Unicode standards. Our findings indicate that the local neighbourhood communication principle is preserved, enabling the ECA to be considered heterogeneous to learn the behaviour of the dataset. The proposed CAML classifier is good enough to predict the class labels of the dataset. In the future, we plan to develop a language-independent generic CAML model for analyzing the toxicity of comments of any language.

References

1. Aken, B.V., Risch, J., Krestel, R., Löser, A.: Challenges for toxic comment classification: an in-depth error analysis. In: Proceedings of the Second Workshop on Abusive Language Online (ALW2), pp. 33–42. Association for Computational Linguistics (2018)
2. Andrii, S.: Hate speech and offensive language dataset (2020). <https://www.kaggle.com/datasets/mrmorj/hate-speech-and-offensive-language-dataset>
3. Badjatiya, P., Gupta, S., Gupta, M., Varma, V.: Deep learning for hate speech detection in tweets. In: Proceedings of the 26th International Conference on World Wide Web Companion, pp. 759–760. International World Wide Web Conferences Steering Committee (2017). <https://doi.org/10.1145/3041021.3054223>
4. Bashar, M.A., Nayak, R.: Qutnocturnal@hasoc'19: CNN for hate speech and offensive content identification in Hindi language (2020). [arXiv:abs/2008.12448](https://arxiv.org/abs/2008.12448)
5. Burnap, P., Williams, M.L.: Cyber hate speech on Twitter: an application of machine classification and statistical modeling for policy and decision making. *Policy Internet* **7**(2), 223–242 (2015)
6. Burnap, P., Williams, M.L.: Us and them: identifying cyber hate on Twitter across multiple protected characteristics. *EPJ Data Sci.* **5**, 1–15 (2016)
7. Cjadams, J.S. et al.: Toxic comment classification challenge (2017). <https://kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge>
8. Davidson, T., Warmlesley, D., Macy, M., Weber, I.: Automated hate speech detection and the problem of offensive language. In: Proceedings of the International AAAI Conference on Web and Social Media, vol. 11, pp. 512–515 (2017)
9. Davidson, T., Warmlesley, D., Macy, M., Weber, I.: Automated hate speech detection and the problem of offensive language. In: Proceedings of the Eleventh International AAAI Conference on Web and Social Media (ICWSM 2017). Association for the Advancement of Artificial Intelligence (2017)
10. Del Vigna¹², F., Cimino²³, A., Dell'Orletta, F., Petrocchi, M., Tesconi, M.: Hate me, hate me not: Hate speech detection on Facebook. In: Proceedings of the First Italian Conference on Cybersecurity (ITASEC17), pp. 86–95 (2017)
11. Elizabeth, M.J., Chaudhuri, P.P., Hazari, R.: Cellular automata-based sentiment analysis for bipolar classification of reviews. *Complex Syst.* (2024)
12. Elizabeth, M.J., Hazari, R.: Cellular automata-based sentiment analysis using deep learning methods. *Complex Syst.* (2024)

13. Elizabeth, M.J., Kommineni, A.K., Hazari, R.: Sentiment analysis for code-mixed data using cellular automata with deep learning models. In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 14978 LNCS, pp. 163–176 (2024). https://doi.org/10.1007/978-3-031-71552-5_14
14. Elizabeth, M., Panda, A.K., Chaudhuri, P.P., Hazari, R.: Cellular automata-based sentiment analysis. In: *Asian Symposium on Cellular Automata Technology*, pp. 53–64. Springer (2023)
15. Elizabeth, M., Parsotambhai, S.M., Hazari, R.: Cellular automata enhanced machine learning model for toxic text classification. In: *International Conference on Cellular Automata for Research and Industry*, pp. 346–355. Springer (2022)
16. Fortuna, P., Nunes, S.: A survey on automatic detection of hate speech in text. *ACM Comput. Surv. (CSUR)* **51**(4), 1–30 (2018)
17. Garg, P., Sharma, M.K., Kumar, P.: Improving hate speech classification through ensemble learning and explainable AI techniques. *Arab. J. Sci. Eng.* (2024). <https://doi.org/10.1007/s13369-024-09540-2>
18. Georgakopoulos, S.V., Tasoulis, S.K., Vrahatis, A.G., Plagianakos, V.P.: Convolutional neural networks for toxic comment classification. In: *Proceedings of the 10th Hellenic Conference on Artificial Intelligence*, pp. 1–6 (2018)
19. Greevy, E.: Automatic text categorisation of racist webpages. Ph.D. thesis, Dublin City University (2004)
20. Gutowitz, H.A.: *Cellular Automata: Theory and Experiment*. MIT Press, Cambridge, MA (1990)
21. J, A.K., S, A., Trueman, T.E., Cambria, E.: Comment toxicity detection via a multichannel convolutional bidirectional gated recurrent unit. *Neurocomputing* **441**, 272–278 (2021). <https://doi.org/10.1016/j.neucom.2021.02.023>
22. Karim, M.R., Chakravarti, B.R., P. McCrae, J., Cochez, M.: Classification benchmarks for under-resourced Bengali language based on multichannel convolutional-LSTM network. In: *7th IEEE International Conference on Data Science and Advanced Analytics (IEEE DSAA,2020)*. IEEE (2020)
23. Mamun, M.B., Tsunakawa, T., Nishida, M., Nishimura, M.: Hate speech detection by using rationales for judging sarcasm. *Appl. Sci. (Switzerland)* **14** (2024). <https://doi.org/10.3390/app14114898>
24. Mandl, T., Modha, S., Shahi, G.K., Jaiswal, A.K., Nandini, D., Patel, D., Schäfer, J.: Overview of the HASOC track at fire 2020: Hate Speech and Offensive Content Identification in Indo-European Languages Under Creative Commons License Attribution 4.0 International (cc by 4.0) (2020). <http://ceur-ws.org>
25. Nandini, D., Schmid, U.: Explaining hate speech classification with model-agnostic methods (2022)
26. Neumann, J.V.: *The Theory of Self-reproducing Automata*, A. W. Burks ed. University of Illinois Press, Urbana and London (1966)
27. Pavlopoulos, J., Malakasiotis, P., Androutsopoulos, I.: Deeper attention to abusive user content moderation, pp. 1125–1135 (2017). <https://doi.org/10.18653/v1/D17-1117>
28. Rajput, G., Pun, N.S., Sonbhadra, S.K., Agarwal, S.: Hate speech detection using static BERT embeddings. In: *Srirama, S.N., Lin, J.C.-W., Bhatnagar, R., Agarwal, S., Reddy, P.K. (eds.) BDA 2021. LNCS, vol. 13147*, pp. 67–77. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-93620-4_6
29. Romim, N., Ahmed, M., Islam, M.S., Sen Sharma, A., Talukder, H., Amin, M.R.: BD-SHS: A benchmark dataset for learning to detect online Bangla hate speech in

- different social contexts. In: Proceedings of the Thirteenth Language Resources and Evaluation Conference, pp. 5153–5162. European Language Resources Association, Marseille, France (2022)
30. Romim, N., Ahmed, M., Talukder, H., Saiful Islam, M.: Hate speech detection in the Bengali language: a dataset and its baseline evaluation. In: Proceedings of International Joint Conference on Advances in Computational Intelligence: IJCACI 2020, pp. 457–468. Springer (2021)
 31. Schmidt, A., Wiegand, M.: A survey on hate speech detection using natural language processing. In: Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media, pp. 1–10 (2017)
 32. Singh, I., Goyal, G., Chandel, A.: AlexNet architecture based convolutional neural network for toxic comments classification. *J. King Saud Univ. - Comput. Inf. Sci.* **34**, 7547–7558 (2022). <https://doi.org/10.1016/j.jksuci.2022.06.007>
 33. Velankar, A., Patil, H., Gore, A., Salunke, S., Joshi, R.: Hate and offensive speech detection in Hindi and Marathi. *CoRR* abs/2110.12200 (2021)
 34. Wang, K., Yang, J., Wu, H.: A survey of toxic comment classification methods. *CoRR* (2021)
 35. Wolfram, S.: Theory and Applications of Cellular Automata. World Scientific, Singapore (1986)
 36. Yin, D., Xue, Z., Hong, L., Davison, B.D., Kontostathis, A., Edwards, L.: Detection of harassment on web 2.0. In: Proceedings of the Content Analysis in the WEB, vol. 2, pp. 1–7 (2009)
 37. Zhao, Z., Zhang, Z., Hopfgartner, F.: A comparative study of using pre-trained language models for toxic comment classification. In: Companion Proceedings of the Web Conference 2021, pp. 500–507. Association for Computing Machinery, Inc (2021). <https://doi.org/10.1145/3442442.3452313>